

Functional & Technical Documentation

Script-Controlled ACL – Restrict Record Access Based on Field Value

1. Introduction

Project Title: Script-Controlled ACL – Restrict Record Access Based on Field Value

Team Members: Jenifer.N , Sowmiya.A, Sujitha Blessy.P ,Yazhini.N ,Priya Dharshini.P

Platform: ServiceNow

2. Project Overview

2.1 Purpose

To restrict record viewing according to the Branch field while retaining full administrator access.

2.2 Key Features

- Script-controlled ACL
- Branch-based record visibility
- EEE-user access to EEE records
- Unauthorized-user denial
- Administrator full access
- Record-level authorization
- Testable and extensible security logic

3. Architecture

3.1 Platform Architecture

ServiceNow provides the web interface, user/role context, application records and ACL evaluation. When a protected record is requested, the ACL engine evaluates the configured script before allowing the user to view the record.

3.2 Authorization Architecture

Input	Evaluation	Decision
Current user is administrator	Administrator exception	ALLOW
Current user is EEE and record Branch = EEE	Branch condition satisfied	ALLOW
Current user does not satisfy branch condition	Restriction applies	DENY

3.3 Data Model

Entity / Field	Purpose
User	Identifies the logged-in user.
Role	Determines whether the user has administrator-level access.
User Branch	Provides the user's branch context, where configured.
Target Record	Record being requested.
Branch	Field on the target record used by the ACL condition.

4. Setup Instructions

4.1 Prerequisites

- Access to a ServiceNow development/lab instance.
- Permission to create or configure ACLs.
- A target table containing a Branch field.
- Test accounts representing EEE, non-EEE and administrator users.
- Sample records with EEE and other Branch values.

4.2 Configuration Steps

1. Open the target ServiceNow table and identify the record-view/read operation that must be protected.
2. Create or open the relevant ACL for the table/record access.
3. Configure the ACL so the scripted logic is executed for the protected access operation.
4. Identify the actual user branch field and administrator role used by the instance.
5. Implement the policy: administrator → allow; otherwise EEE user + record Branch = EEE → allow; otherwise → deny.
6. Save and activate the ACL.
7. Create/select EEE and non-EEE records and test using the designated user accounts.
8. Record the results and correct any unexpected authorization behaviour.

5. Folder Structure

Not applicable in the same sense as a MERN client/server project. The implementation is primarily stored as ServiceNow configuration and server-side ACL script.

ServiceNow Artifact	Purpose
ACL record	Stores access-control configuration and script.
Application table	Stores protected records.
User records	Stores user identity and authorization context.
Role configuration	Determines administrator privileges.
Update Set / Application scope	Optional mechanism for moving configuration between instances.

6. Running / Executing the Application

There is no separate client/server start command. The solution runs inside the ServiceNow instance. After the ACL is activated, access-control logic is invoked when a user requests the protected record operation.

7. API Documentation

No custom REST API is required by the described solution. The core behaviour is implemented through ServiceNow's ACL mechanism and server-side script execution. If an external API is added later, document its method, endpoint, parameters and response here.

8. Authentication & Authorization

Authentication is handled by the ServiceNow platform. Authorization for the protected record is handled by the configured ACL. The ACL uses the logged-in user's context, administrator role and Branch-related values to decide whether the record may be viewed. Credentials must not be hard-coded in the ACL script.

9. User Interface

The project uses the standard ServiceNow list and form interfaces. Insert screenshots of the EEE-user allowed view, unauthorized-user denial, and administrator access in the final report.

10. Testing

Testing covers functional positive/negative scenarios, branch-field edge cases, repeated access consistency and basic performance observation. The expected policy is:

- EEE user + EEE record → Allow
- Non-EEE user + EEE record → Deny
- Administrator + EEE record → Allow
- Administrator + non-EEE record → Allow

11. Screenshots or Demo

- ACL configuration screen
- EEE user successful access
- Unauthorized user denied access
- Administrator successful access
- Test evidence / final result

Demo Link: _____

12. Known Issues

- Actual user branch field name and administrator role are instance-specific and must be confirmed before deployment.
- Incorrect ACL ordering or conflicting ACLs may affect the observed result.
- Empty or unexpected Branch values require explicit testing according to the final policy.

13. Future Enhancements

- Support multiple branches through configurable mappings.
- Support more granular roles and record attributes.
- Add automated regression testing.
- Add audit/reporting for access decisions where supported.
- Package the ACL pattern for reuse on other sensitive tables.